SIMATS ENGINEERING
ASSIGNMENT 1

COURSE NAME: DATA STRUCTURE
COURSE CODE: CSA0307
NAME / REG NO: MONISH.L [1972260035]

Q1. Define structure in C and its role in data structures (5 Marks)

Answer:

A **structure** in C is a user-defined data type, declared with the keyword `struct`, that groups a fixed number of variables (called members) of **different data types** under one name. Each member occupies its own memory and is accessed with the dot operator (`s.name`) or, through a pointer, with the arrow operator (`p->name`).

Syntax and example

```
struct Student {
    int   roll;
    char  name[20];
    float marks;
};

struct Student s = {1, "Asha", 87.5f};
printf("%d %s %.1f", s.roll, s.name, s.marks);   /* 1 Asha 87.5 */
```

Role of structures in data structures

- **Representing a record:** an array can hold only elements of one type, but a real item (student, employee, process) has several attributes of different types. A structure stores them together as one unit, and an array of structures forms a table of records.
- **Building nodes:** a structure can contain a pointer to a structure of its own type (a self-referential structure). This is the basis of dynamic data structures: each node of a linked list, stack, queue, tree or graph is a structure holding the data and one or more links, e.g. `struct Node { int data; struct Node *next; };`.
- **Abstraction:** a stack or queue is written as a structure that bundles its array, `top`/`front`/`rear` and size, so one variable represents the whole data structure and can be passed to functions.
- **Dynamic memory:** nodes are created at run time with `malloc(sizeof(struct Node))`, so the structure grows and shrinks as needed, unlike a fixed-size array.

```
struct Node { int data; struct Node *next; };  /* self-referential */
/* list of three nodes:  10 -> 20 -> 30 -> NULL */
```

Conclusion: structures let C programs combine related data and links into a single unit, and they are the building block from which linked lists, stacks, queues, trees and graphs are constructed.

Q2. Compare memory usage of recursion vs iteration (10 Marks)

Answer:

Recursion solves a problem by a function calling itself on a smaller input, while **iteration** repeats a block using a loop (`for`, `while`). Both can compute the same result, but they use memory very differently.

How memory is used

- **Recursion:** every call creates a new **stack frame** on the call stack holding the parameters, local variables, return address and saved registers. All frames of a chain stay in memory until the base case is reached and the calls return one by one. For depth n the space is **O(n)**.
- **Iteration:** the loop reuses the same variables in a single stack frame; only a few variables (counter, accumulator) are stored. The extra space is **O(1)**.

```
long fact_rec(int n) {              /* O(n) stack space */
    return n <= 1 ? 1 : n * fact_rec(n - 1);
}
long fact_iter(int n) {             /* O(1) space */
    long r = 1;
    for (int i = 2; i <= n; i++) r *= i;
    return r;
}
```

For `n = 5` both return 120. `fact_rec(5)` creates the frames fact_rec(5), (4), (3), (2), (1) – five frames alive at the same time at the deepest point – while `fact_iter(5)` uses one frame with the variables `r` and `i`.

Comparison

Basis	Recursion	Iteration
Memory (space)	O(n) – one stack frame per call (for factorial, linear search, list traversal)	O(1) – a fixed number of variables in one frame
Where stored	Call stack (limited, typically a few MB)	Same frame; no extra frames
Overhead	Each call pushes/pops a frame: extra time and memory	No call overhead; usually faster
Risk	Very deep or infinite recursion causes **stack overflow**	Infinite loop wastes time but does not exhaust the stack
Naive Fibonacci	O(n) stack depth and repeated subproblems (exponential time)	O(1) space with two variables, linear time
Tree/graph traversal, quick sort	Natural and short; stack use equals tree depth	Needs an explicit stack, which uses heap/array memory of similar size
Code size	Short and readable	Often longer

Tail recursion (the recursive call is the last operation) can be optimized by some compilers to reuse the frame and reach O(1) space, but C does not guarantee this, so it should not be relied on.

Conclusion: iteration is more memory-efficient (O(1) auxiliary space versus O(n) for recursion) and is preferred when the depth can be large. Recursion costs extra stack memory, but is justified when it makes the solution clearer, as in tree traversals, divide-and-conquer and backtracking.

Q3. Explain use of linear structures in scheduling systems (5 Marks)

Answer:

A **linear data structure** stores elements in a sequence, each element having one predecessor and one successor (array, linked list, stack, queue). Operating systems and other scheduling systems must keep processes or tasks in a definite order, so linear structures are used directly.

- **Queue (FIFO) as the ready queue:** processes waiting for the CPU are kept in a queue; the process at the front is dispatched and new or pre-empted processes join at the rear. First-Come-First-Served (FCFS) scheduling is exactly a queue.

- **Circular queue in Round Robin:** each process runs for a time quantum; if it is not finished it goes to the rear of the queue. A circular array reuses freed slots, so no space is wasted. Example with burst times P1=5, P2=3, P3=8, P4=2 and quantum 3: the order of execution is P1, P2, P3, P4, P1, P3, P3, finishing at time 18.
- **Linked list for dynamic queues:** the number of processes changes continuously, so a linked list (e.g. the process control block list) allows insertion and deletion at any position without shifting elements, which is also used for sorted queues such as Shortest Job First or priority-ordered lists.
- **Multi-level queues:** separate queues (system, interactive, batch) are kept for different priority classes, each being a linear structure scheduled by its own policy.
- **Stack (LIFO):** used for the context saved on interrupts and for nested task or function calls, so the most recent task is resumed first.
- **Deque and job queues:** printers, disk and network schedulers keep requests in queues and serve them in arrival order (or sorted order, as in SSTF/SCAN disk scheduling).

```
/* Round Robin ready queue as a circular queue (quantum = 3) */
enqueue(p);   p = dequeue();   run(p, quantum);
if (burst[p] > 0) enqueue(p);   /* unfinished: back to rear */
```

Conclusion: because scheduling is about order (who goes first, who waits), queues, lists and stacks are the natural structures; they give simple, constant-time insertion and removal at the ends and make the scheduling policy easy to implement.

Signature of the student:

